

Rapport | IFT-2035

Tarik Hireche : 202 301 89 | Mohammed Kamal Skhyri : 202 283 77

16. November 2025

Rapport écrit par Tarik Hireche

1 Implementation du code

Pour l'allocateur à pile, nous avons défini la structure suivante :

```
1 const AllocateurPile = struct {  
2     buffer: []u8,  
3     next: usize,  
4 };
```

zig

La fonction `alloc` est responsable de calculer le bon alignement avec :

```
1 const align_offset = std.mem.alignForward(usize, current,  
    alignment.toByteUnits());
```

zig

et d'avancer le pointeur `next` après chaque allocation réussie. L'allocation échoue si `start + len > buffer.len`. Ce comportement reproduit celui d'une pile : les blocs sont empilés sans jamais être libérés individuellement.

Pour l'allocateur à étiquette, nous avons ajouté une structure `Header` :

```
1 const Header = struct {  
2     len: usize,  
3     free: bool,  
4 };
```

zig

Chaque allocation réserve de l'espace pour un `Header`, place juste avant les données réelles. L'adresse de début des données est calculée en utilisant l'adresse absolue du buffer (`base_addr`) pour garantir la portabilité (notamment sur macOS/ARM où la pile n'est pas toujours alignée) :

```
1 const base_addr = @intFromPtr(self.buffer.ptr);  
2 const current_addr = base_addr + self.next;  
3  
4 const header_addr = std.mem.alignForward(usize, current_addr,  
    header_alignment.toByteUnits());  
5 const data_addr = std.mem.alignForward(usize, header_addr + @sizeOf(Header),  
    alignment.toByteUnits());
```

zig

Ensuite, on écrit le `Header` en mémoire, on met `free = false` et on retourne le pointeur vers `data_start`. La fonction `free` ne libère pas physiquement le bloc, mais met simplement `header.free = true`, et `getHeader` récupère le header à partir du pointeur donné.

2 Difficultés et choix techniques

2.1 L'alignement mémoire : un concept central

Bon, l'un des premiers défis a été la gestion de l'alignement mémoire, un aspect essentiel en Zig. Chaque type (`u8`, `u16`, `u32`, `u64`, `struct`, etc.) doit commencer à une adresse multiple de son alignement naturel. Si ce n'est pas respecté, Zig refuse l'allocation ou signale une erreur.

Pour garantir des allocations valides, nous avons utilisé :

```
1 const align_offset = std.mem.alignForward(usize, current,
    alignment.toByteUnits());
```

zig

Cela nous permettait d'aligner systématiquement l'adresse de départ de chaque bloc, ce qui rend l'allocation fiable et conforme aux exigences matérielles.

Dans l'allocateur à étiquette, la difficulté était double : il fallait aligner le `Header` et les données juste après. Nous avons donc calculé deux positions distinctes :

```
1 const header_start = std.mem.alignForward(usize, self.next,
    header_alignment.toByteUnits());
2 const data_start = std.mem.alignForward(usize, header_start + @sizeOf(Header),
    alignment.toByteUnits());
```

zig

Cette approche garantit une mémoire bien structurée, avec un header et des données chacun correctement alignés.

2.2 Manipulation des pointeurs et des slices

En Zig, une différence importante existe entre une slice (`[]u8`) et un pointeur brut (`[*]u8`). Notre fonction `alloc` devait renvoyer un pointeur brut, mais une tranche (`buffer[start ..]`) produit une slice.

Après quelques essais, la solution correcte était :

```
1 return self.buffer.ptr + start;
```

zig

Ce retour donne bien un `[*]u8`, type attendu par `std.mem.Allocator`.

Pour retrouver le `Header` d'un bloc (dans `getHeader`), nous avons utilisé l'arithmétique de pointeurs typés pour reculer de la taille du header, ce qui évite les casts hasardeux via des entiers :

```
1 const raw_ptr = ptr - @sizeOf(Header);
2 return @ptrCast(@alignCast(raw_ptr));
```

zig

Cette opération nous a permis de mieux comprendre comment Zig représente la mémoire : tout passe par des conversions contrôlées, explicites et strictement typées.

2.3 Structure et décisions d'implémentation

Notre travail s'est construit autour d'une logique d'évolution progressive : chaque allocateur introduit un nouveau concept fondamental de gestion mémoire; la pile, l'étiquette, puis le recyclage.

Nous avons renforcé notre implémentation avec plusieurs tests additionnels :

- allocations répétées (stress test),
 - alignements variés
 - cas proches de la saturation du buffer.
-

Exemples concrets

1. Allocateur à pile :

Supposons un buffer de 16 octets. On effectue successivement trois allocations de 4 octets :

1. `alloc(u8, 4)` → occupe les cases 0 à 3
2. `alloc(u8, 4)` → occupe les cases 4 à 7
3. `alloc(u8, 4)` → occupe les cases 8 à 11

Si on tente une quatrième allocation, elle échoue car `next` atteindrait 16 (taille du buffer).

Aucune libération n'est possible individuellement, ce qui illustre bien le comportement "empilé" : **les blocs sont ajoutés les uns après les autres sans jamais être retirés.**

2. Allocateur à étiquette :

Ici, chaque bloc est précédé d'un petit "en-tête" (**Header**) qui contient deux informations :

1. la taille du bloc (`len`)
2. et son état (`free`).

Ce **Header** est stocké juste avant les données à une adresse alignée selon le type **Header** pour éviter tout accès invalide.

Exemple : si on alloue 8 octets pour un `u64`, l'allocateur calcule d'abord l'adresse alignée du **Header**, écrit `{ len = 8, free = false }` puis réserve les 8 octets de données immédiatement après.

Cette étape **garantit** que le **Header** et les données sont chacun **correctement alignés en mémoire**.

Lorsqu'on appelle `free()`, le **Header** est retrouvé en mémoire à partir du pointeur du bloc (en reculant de `@sizeof(Header)` octets).

Le champ `free` passe alors à `true`. Aucune réutilisation de mémoire n'est faite pour l'instant. On se contente de marquer le bloc comme libre, ce qui prépare la suite du devoir, où l'allocateur "recycleur" viendra réemployer ces zones libérées.

(Conformément à ce que notre professeur Mattéo a dit sur StudiuM, **on suppose ici que les alignements demandés sont inférieurs ou égaux à 8 octets**, ce qui simplifie le calcul du padding entre le Header et les données.)

Ces tests ont confirmé la stabilité de `self.next` et l'absence de dépassements ou de corruption mémoire.

3. Allocateur à recyclage :

Pour clôturer notre implémentation, nous avons développé la troisième version : `AllogRe-tractor`, notre allocateur à recyclage complet.

L'objectif : **réutiliser les blocs libérés** (`free = true`) plutôt que d'empiler indéfiniment de nouvelles allocations.

La structure de base reste la même :

```
1 const Header = struct { zig
2     len: usize,
3     free: bool,
4 };
```

L'idée clé en soit est simple :

Avant d'écrire un nouveau bloc à la fin du buffer, on parcourt toute la mémoire déjà allouée à la recherche d'un Header libre. Si on trouve un bloc dont la taille (`len`) est suffisante, on le réactive immédiatement :

```
1 var offset: usize = 0; zig
2 while (offset < self.next) {
3     const base_addr = @intFromPtr(self.buffer.ptr);
4     const header_addr = base_addr + offset;
5     const header_ptr: *Header = @ptrFromInt(header_addr);
6
7     if (header_ptr.free and header_ptr.len >= len) {
8         header_ptr.free = false;
9         const data_addr = header_addr + @sizeOf(Header);
10        return @ptrFromInt(data_addr);
11    }
12
13    offset += @sizeOf(Header) + header_ptr.len;
14 }
```

Si aucun bloc libre ne correspond, alors on poursuit normalement, c'est à dire:

On place un nouveau Header à la fin du buffer et on avance `self.next`.

Grâce à ça, **le recyclage devient automatique!** Aucune réallocation coûteuse, aucune perte d'espace.

Je réitère que pour garantir la compatibilité multi-plateforme, on a suivi la précision de Mattéo sur StudiumM:

"On suppose que tous les alignements demandés sont inférieurs ou égaux à 8 octets."

- Mattéo

Cela simplifie énormément le calcul du padding entre le Header et les données.

La fonction free reste assez élégante:

```
1 const header = getHeader(buf.ptr);  
2 header.free = true;
```

zig

Ce marquage suffit à rendre la zone réutilisable lors d'un prochain appel à `alloc()`.

Réflexion finale

Ce dernier allocateur marque la fin logique du parcours : on passe d'un modèle linéaire (pile), à un modèle étiqueté, puis à un modèle intelligent et recyclable. Ce projet nous a forcés à raisonner comme le ferait un vrai gestionnaire mémoire du système : comprendre les alignements, maîtriser les pointeurs et éviter la corruption d'état.

Au final, voir les tests passés sur deux OS différents a été une des plus belles satisfactions du devoir haha!

3 Démarche et compréhension du travail

Avant de se lancer dans le code, on a pris le temps de bien comprendre ce que Zig voulait vraiment dire par **allocator**. C'est pas juste un malloc déguisé : chaque allocateur a sa propre logique interne, ducoup Zig oblige à la définir soi-même.

On a donc commencé par étudier `std.heap.page_allocator` pour piger la structure `std.mem.Allocator` et son protocole (`alloc`, `free`, etc.).

Ensuite, on a monté notre implémentation étape par étape :

- d'abord une version minimale de l'allocateur à pile, juste pour tester la mécanique d'allocation linéaire ;
 - Avant d'ajouter la gestion d'alignement, il fallait
- puis on a ajouté la gestion d'alignement pour rendre tout ça propre et conforme aux règles matérielles ;
- enfin, on a étendu le modèle avec les **étiquettes** (headers) pour stocker des infos par bloc et préparer la suite du devoir (le recycleur).

Cette approche incrémentale nous a permis de valider chaque composante séparément avec `zig test`, et de corriger les comportements au fur et à mesure.

4 Tests et validation

On a écrit plusieurs tests pour valider que notre allocateur réagit bien dans toutes les situations :

- des allocations successives pour vérifier que `self.next` évolue correctement ;
- des tests d'alignement (par exemple `u8`, `u16`, `u64`) pour s'assurer que chaque bloc tombe sur une adresse multiple de son alignement ;
- un test de dépassement pour confirmer qu'une demande trop grosse renvoie bien `null` ou `error.OutOfMemory`.

Exemple concret : en allouant successivement des `u8`, `u32`, puis `u64`, on a vérifié avec `@intFromPtr` que chaque bloc commence à la bonne adresse et qu'aucun octet n'est écrasé. Même après plusieurs dizaines d'allocations, le curseur `next` restait cohérent et aucun test n'a échoué.

5 Sources utilisées pour ce devoir:

- [source I – Documentation officielle de Zig \(std.mem\)](#)
- [source II – Exemples d'allocateurs dans la librairie standard](#)
- [source III – Tutoriel Ziggит.dev sur les allocateurs](#)
- [source IV – Learn Zig : mémoire, pointeurs et alignements](#)
- [source V – Notes de cours IFT2035 sur la mémoire et les allocateurs](#)
- source VI – ChatGPT utilisé uniquement pour des clarifications techniques et certains bouts de codes; la grande majorité du code, l'entièreté des tests et les décisions d'implémentation ont été réalisés par nous.